



National University
Of Computer and Emerging Sciences

PDC Final Project

Single Source Shortest Path Algorithm

Salman Khan (22i-1285)

Mavra Mehak (22i-0798)

Maryum Tanvir (22i-0751)

CS-E

Introduction

This project implements a **Dynamic Single Source Shortest Path (SSSP)** solver designed to efficiently handle large and evolving graphs. Traditional SSSP algorithms like Dijkstra's recompute the entire shortest path tree from scratch whenever the graph changes, which is computationally expensive and impractical for dynamic or real-time systems. To address this, our solution combines **SSSP initialization** with **parallel dynamic updates** using **OpenMP**, **MPI** and **METIS** enabling scalable and responsive shortest path computation after edge insertions and deletions.

The system is built to read real world graph data and perform **batch updates** on the graph structure while maintaining correctness and minimizing recomputation. By identifying only the affected subgraph during updates, and applying **asynchronous, lock-free updates in parallel**, the solution significantly reduces runtime on large datasets. This makes it well-suited for applications such as **network routing**, **transportation planning**, **social network analysis**, and any domain where fast response to dynamic changes in graph topology is critical.

Parallelization Methods Used

Reason for Different Versions

The implementation of the dynamic Single-Source Shortest Path (SSSP) algorithm was developed in multiple variants to address distinct performance, scalability, and assignment requirements. The sequential implementation served as a baseline for performance analysis, providing a reference to evaluate parallel implementations by eliminating concurrency overheads. The OpenMP-only implementation was included to align with the approach described in the reference paper. The MPI implementation without METIS was developed to avoid METIS's limitations in handling edge partitioning, which can lead to incorrect edge assignments across partitions. Finally, the MPI with METIS implementation was implemented as per the project's directive to explore graph partitioning, aiming to minimize inter-process communication by assigning connected subgraphs to processes, despite challenges with METIS's contiguous partitioning on non-contiguous graphs. These implementations collectively enabled a comprehensive evaluation of the algorithm's performance across sequential, shared-memory, and distributed-memory environments.

OpenMP

OpenMP significantly improves performance for large scale graph problems like dynamic SSSP by parallelizing computationally intensive tasks such as edge updates, affected node propagation, and shortest path recalculations. In our implementation, OpenMP is used in batch processing edge insertions/deletions and in iterative update phases where redundant computations are allowed. This parallelism reduces the time complexity of these phases by distributing work across available CPU cores, making it ideal for workloads with high vertex and edge counts—where sequential algorithms would bottleneck due to their iterative nature

MPI

MPI (Message Passing Interface) is used in this project to achieve **inter-node parallelization**, enabling the distribution of graph processing tasks across multiple computing nodes in a cluster. Since large-scale graphs often exceed the memory and processing capabilities of a single machine, MPI facilitates dividing the graph into subgraphs—each assigned to a separate node—allowing concurrent computation of the SSSP problem. Communication between nodes is handled via message passing, ensuring that updates from affected vertices in one partition are efficiently propagated to neighboring partitions. This parallel model significantly enhances scalability and performance for massive, dynamic graphs by leveraging distributed memory systems and minimizing idle computation across the cluster.

Performance Analysis

Dynamic_MP.cpp Profile (gprof)

This report analyzes the performance profile of `Dynamic_MP.cpp`, an OpenMP-based parallel implementation of a dynamic Single-Source Shortest Path (SSSP) algorithm, as provided in `openmp_profile.txt`. The profile, generated using `gprof`, details the runtime behavior, highlighting key functions and their computational costs. The analysis focuses on the flat profile, call graph, and metrics to identify bottlenecks, assess parallelization effectiveness, and suggest optimizations, with comparisons to the sequential implementation (`Sequential.cpp`).

1. Overview of Profiling Data

The gprof output includes:

- **Flat Profile:** Lists functions by self-time (time spent in the function itself) and cumulative time (including callees).
- **Call Graph:** Shows the function call hierarchy, including callers, callees, and time propagation.

Key Metrics:

- **% Time:** Percentage of total program runtime (56.30 seconds) spent in a function.
- **Self Seconds:** Time spent executing the function's own code.
- **Cumulative Seconds:** Total time including the function and its callees.
- **Calls:** Number of invocations.
- **Self ms/call:** Average time per call for the function's own execution.
- **Total ms/call:** Average time per call including callees.
- **Name:** Function name, often with complex C++ template signatures.

2. Flat Profile Analysis

The flat profile ranks functions by self-time, identifying the most computationally intensive operations.

Top Time-Consuming Functions:

1. **loadGraph (60.39%, 34.00 seconds):**
 - **Description:** Loads and preprocesses the graph from a file, constructing the adjacency list and handling duplicate edges.
 - **Analysis:** Dominates runtime, similar to the sequential version (58.55%, 35.00 seconds). The high time is due to reading and processing a large input file (graph.txt), with significant overhead in duplicate edge detection.
 - **Calls:** 22 (notably higher than the sequential version's 1 call, likely due to repeated calls within parallelSSSPUpdate).
 - **Self s/call:** 1.55 seconds per call.
 - **Total s/call:** 2.44 seconds, including child functions.
2. **std::insert_unique (33.27%, 18.73 seconds):**
 - **Description:** Part of std::set used in loadGraph and loadChanges for duplicate edge detection (edge_set.insert, change_set.insert).
 - **Analysis:** Accounts for a significant portion of loadGraph's time, with 10,000,042 calls reflecting edge insertions. The logarithmic complexity of tree operations is a bottleneck, consistent with the sequential version (31.48%, 18.82 seconds).
 - **Calls:** 10,000,042 (10M from loadGraph, 42 from loadChanges).

- **Self s/call:** Negligible per call, but cumulative due to high call volume.
- 3. **frame_dummy (3.59%, 2.02 seconds):**
 - **Description:** A compiler-generated function, often related to profiling or initialization.
 - **Analysis:** The 5,494,413 calls indicate potential gprof instrumentation overhead, similar to the sequential version (3.08%, 1.84 seconds). This suggests profiling artifacts inflating times.
 - **Calls:** 5,494,413, unusually high.
- 4. **sequentialSSSP (2.01%, 1.13 seconds):**
 - **Description:** Implements Dijkstra's algorithm to compute the initial SSSP tree.
 - **Analysis:** Slightly slower than the sequential version (1.79%, 1.07 seconds), possibly due to differences in graph size or compilation settings. The single call processes the entire graph.
 - **Calls:** 1.
 - **Total s/call:** 2.51 seconds, including children.
- 5. **void std::vector<std::pair<int, int>, std::allocator<std::pair<int, int>>> >::_M_realloc_insert (0.57%, 0.32 seconds):**
 - **Description:** Vector reallocation for adjacency list construction in loadGraph and sequentialSSSP.
 - **Analysis:** Indicates dynamic vector growth, with 5,005,047 calls reflecting frequent resizing. Slightly lower time than the sequential version (0.72%, 0.43 seconds) due to smaller integer types (int vs. int64_t).
 - **Calls:** 5,005,047.
- 6. **parallelSSSPUpdate (0.00%, 0.00 seconds):**
 - **Description:** Parallel implementation of dynamic SSSP updates using OpenMP, processing edge insertions and deletions in batches.
 - **Analysis:** Surprisingly low self-time, but the call graph shows 51.25 seconds in children, primarily loadGraph (32.45 self + 18.79 children). This suggests parallelSSSPUpdate indirectly triggers costly operations, possibly due to repeated graph loading or inefficient parallelization.
 - **Calls:** 1.
 - **Total s/call:** 51.25 seconds, reflecting significant child contributions.
- 7. **loadChanges (0.04%, 0.02 seconds):**
 - **Description:** Reads edge changes from update_small.txt.
 - **Analysis:** Minimal time, indicating a small update file or efficient parsing, comparable to readChangesFromFile in the sequential version (0.00%, 0.00 seconds).
 - **Calls:** 1.
 - **Self s/call:** 0.02 seconds.

Summary:

- **Total Runtime:** 56.30 seconds (slightly faster than sequential's 59.78 seconds).
- **Dominant Function:** loadGraph (60.39%) and its child std::insert_unique (33.27%) account for ~93% of runtime, mirroring the sequential version's bottleneck.
- **Parallel Component:** parallelSSSPUpdate has negligible self-time but significant child time, suggesting profiling limitations of not detecting Open MP.

3. Call Graph Analysis

The call graph illustrates function interactions and time propagation.

Key Observations:

- **Main [1] (99.9%, 0.02 self, 56.24 children):**
 - Calls loadGraph, sequentialSSSP, parallelSSSPUpdate, and loadChanges.
 - Most time is in children, particularly parallelSSSPUpdate (51.25 seconds in children) and loadGraph (1.55 self + 0.89 children).
 - High call count for std::__cxx11::to_string (6,420,370) from output operations.
- **loadGraph [2] (95.4%, 34.00 self, 19.69 children):**
 - Called 22 times (1 from main, 21 from parallelSSSPUpdate), a significant increase from the sequential version's single call.
 - Major children:
 - std::_Rb_tree::_M_insert_unique (18.73 seconds, 10M calls): Edge duplicate detection.
 - frame_dummy (0.64 seconds, 1,737,331 calls): Profiling overhead.
 - Vector reallocation (0.32 seconds, 5,005,025 calls).
 - High time reflects repeated graph loading or reinitialization within parallelSSSPUpdate.
- **parallelSSSPUpdate [3] (91.0%, 0.00 self, 51.25 children):**
 - Surprisingly high child time, with 32.45 seconds in loadGraph (self) and 18.79 seconds in loadGraph's children.
 - Calls loadGraph 21 times, suggesting redundant graph reinitialization or misuse in the parallel update loop.
 - Minimal direct computation (0.00 self), indicating that OpenMP parallel sections are not captured effectively by gprof or are overshadowed by sequential bottlenecks.
- **sequentialSSSP [5] (4.5%, 1.13 self, 1.38 children):**
 - Calls frame_dummy (1.38 seconds, 3,757,078 calls), likely profiling overhead.
 - Minimal other children, consistent with the sequential version.
- **loadChanges [9] (0.04%, 0.02 self):**
 - Calls std::_Rb_tree::_M_insert_unique (42 calls) for change deduplication and vector reallocation (7 calls).

- Low time reflects a small update file (update_small.txt).

4. Performance Bottlenecks and Insights

Primary Bottleneck:

- **Graph Loading** (loadGraph):
 - **Issue:** 60.39% of runtime (34.00 seconds) is spent in loadGraph, with 33.27% (18.73 seconds) in `std::_Rb_tree::_M_insert_unique` for duplicate edge detection.
 - **Cause:** The `set<pair<int, int>>` for deduplication has $O(\log n)$ insertion complexity, exacerbated by 10M edge insertions. The 22 calls to loadGraph (vs. 1 in sequential) suggest redundant graph reinitialization in `parallelSSSPUpdate`.
 - **Impact:** Graph preprocessing overshadows parallel SSSP updates, limiting the benefits of OpenMP.

Secondary Bottlenecks:

- **Profiling Overhead:** `frame_dummy` (3.59%, 2.02 seconds, 5.5M calls) indicates gprof instrumentation issues, similar to the sequential version (3.08%, 1.84 seconds).
- **Memory Management:** Vector reallocations (0.57%, 0.32 seconds for `pair<int, int>`) reflect dynamic resizing in loadGraph, slightly improved over sequential (0.72%, 0.43 seconds) due to smaller data types.
- **Sequential SSSP:** `sequentialSSSP` (2.01%, 1.13 seconds) is efficient but slightly slower than sequential (1.79%, 1.07 seconds), possibly due to graph differences.

Parallel Performance:

- **Issue:** `parallelSSSPUpdate` has negligible self-time (0.00 seconds) but 51.25 seconds in children, primarily loadGraph. This suggests:
 - **Implementation Error:** The 21 calls to loadGraph within `parallelSSSPUpdate` are unexpected, as graph updates should modify the existing graph (`G.addEdge`, `G.removeEdge`) without reloading.
 - **Profiling Limitation:** gprof may not accurately capture OpenMP parallel sections, underreporting time in parallel loops.
- **Impact:** Parallelization benefits are nullified by sequential bottlenecks and potential implementation issues.

Efficient Components:

- **Update Processing:** `loadChanges` (0.04%, 0.02 seconds) and `parallelSSSPUpdate`'s direct computation are fast, indicating that the update set (update_small.txt) is small or updates are efficient when not overshadowed by loadGraph.

- **Output Operations:** Minimal time in `to_string` (0.04%, 0.02 seconds) despite high call counts.

Comparison to Sequential:

- **Runtime:** OpenMP (56.30 seconds) is slightly faster than sequential (59.78 seconds), but the difference (~3.5 seconds) is modest, suggesting limited parallelization gains.
 - **Bottlenecks:** Both versions are dominated by `loadGraph` and `std::_Rb_tree::_M_insert_unique`, with similar percentages (60.39% vs. 58.55%, 33.27% vs. 31.48%).
 - **Parallel Updates:** `parallelSSSPUpdate` (0.00% self, 51.25 seconds children) replaces `updateSSSP` (0.02%, 0.01 seconds), but its high child time indicates inefficiencies not present in the sequential version.
 - **Calls to `loadGraph`:** The OpenMP version's 22 calls vs. sequential's 1 call is a critical difference, pointing to a flaw in `parallelSSSPUpdate`.
-

6. Conclusion

The `Dynamic_MP.cpp` OpenMP implementation's performance is dominated by graph loading (`loadGraph`, 60.39%), particularly duplicate edge detection (`std::_Rb_tree::_M_insert_unique`, 33.27%), mirroring the sequential version (`Sequential.cpp`). The parallel SSSP update (`parallelSSSPUpdate`) shows negligible self-time but high child time (51.25 seconds), primarily due to 21 unexpected calls to `loadGraph`, indicating a likely implementation error. The total runtime (56.30 seconds) is only slightly faster than the sequential version (59.78 seconds), suggesting limited parallelization benefits.

Key Takeaways:

- **Bottleneck:** Redundant `loadGraph` calls and `std::set` insertions in graph preprocessing.
- **Parallelization Issue:** `parallelSSSPUpdate`'s effectiveness is obscured by sequential overhead and potential implementation flaws.
- **Next Steps:**
 - Fix `parallelSSSPUpdate` to eliminate redundant `loadGraph` calls.
 - Replace `set` with `unordered_set` for deduplication.
 - Re-profile with OpenMP-aware tools to assess true parallel performance.
 - Optimize vector allocations and experiment with OpenMP parameters (`batch_size`, `async_level`).

Comparison to Sequential:

- The OpenMP version underperforms expectations due to redundant loadGraph calls, which amplify the same preprocessing bottleneck seen in the sequential version.
- Fixing the implementation error could make parallelSSSPUpdate significantly faster than updateSSSP, especially for larger update sets, leveraging OpenMP's parallelism.

Performance Analysis Report: Sequential.cpp Profile (gprof)

This report analyzes the performance profile of Sequential.cpp generated using gprof, as provided in sequential_profile.txt. The profile offers insights into the runtime behavior of the program, highlighting which functions consume the most computational resources. The analysis focuses on the flat profile, call graph, and key metrics to identify bottlenecks and suggest optimization opportunities.

1. Overview of Profiling Data

The gprof output provides two main sections:

- **Flat Profile:** Shows the time spent in each function, including self-time (time spent in the function itself) and cumulative time (including time in called functions).
- **Call Graph:** Details the function call hierarchy, showing how functions are invoked, their callers, and callees, along with time propagation.

Key Metrics:

- **% Time:** Percentage of total program runtime (59.78 seconds) spent in a function.
- **Self Seconds:** Time spent executing the function's own code.
- **Cumulative Seconds:** Total time including the function and its callees.
- **Calls:** Number of times the function was invoked.
- **Self ms/call:** Average time per call for the function's own execution.
- **Total ms/call:** Average time per call including callees.
- **Name:** Function name, with complex signatures indicating C++ template instantiations.

2. Flat Profile Analysis

The flat profile ranks functions by their self-time, revealing the most time-consuming operations.

Top Time-Consuming Functions:

1. **loadGraph (58.55%, 35.00 seconds):**
 - **Description:** Loads and preprocesses the graph from a file, constructing adjacency lists and CSR format.
 - **Analysis:** Dominates runtime, indicating that graph loading and preprocessing (e.g., reading edges, mapping vertices, removing duplicates) are the primary bottlenecks. The high time is likely due to processing a large input file (large_input.txt).

- **Calls:** 1 (single invocation in main).
 - **Self s/call:** 35.00 seconds, reflecting the entire time spent in this function.
 - **Total s/call:** 57.35 seconds, including time in child functions.
2. **std::tree::insert_unique (31.48%, 18.82 seconds):**
- **Description:** Part of the std::set implementation, used in loadGraph to detect duplicate edges (edge_set.insert).
 - **Analysis:** The high time (10 million calls) suggests that inserting edges into the set<pair<int64_t, int64_t>> is costly, likely due to the logarithmic complexity of tree operations and the large number of edges.
 - **Calls:** 10,000,000, indicating one insertion per edge processed.
 - **Self s/call:** Negligible per call, but cumulative due to volume.
3. **frame_dummy (3.08%, 1.84 seconds):**
- **Description:** A compiler-generated function, often related to initialization or profiling overhead.
 - **Analysis:** The 5,494,414 calls suggest profiling artifacts or linker-related overhead. This time is unexpected and may indicate instrumentation issues.
 - **Calls:** 5,494,414, unusually high for a dummy function.
4. **sequentialSSSP (1.79%, 1.07 seconds):**
- **Description:** Implements Dijkstra's algorithm to compute the initial SSSP tree.
 - **Analysis:** Relatively low time, indicating that the initial SSSP computation is efficient compared to graph loading. The single call processes the entire graph.
 - **Calls:** 1.
 - **Total s/call:** 2.33 seconds, including child functions.
5. **updateSSSP (0.02%, 0.01 seconds):**
- **Description:** Updates the SSSP tree after edge changes.
 - **Analysis:** Minimal time, suggesting that the update file (large_update.txt) contains few changes or that updates are efficient.
 - **Calls:** 1.
 - **Self s/call:** 0.01 seconds.

Other Notable Functions:

- Vector reallocation functions (e.g., _M_realloc_insert for Edge, pair<long, long>) consume small but measurable time (0.21–0.81 seconds), reflecting dynamic memory management overhead in loadGraph.
- std::__cxx11::to_string (0.02%, 0.01 seconds, 6,420,370 calls) is used for output formatting, with negligible per-call time but noticeable due to frequent calls.

Summary:

- **Total Runtime:** 59.78 seconds.
- **Dominant Function:** loadGraph and its child std::insert_unique account for ~90% of runtime, highlighting graph preprocessing as the bottleneck.
- **SSSP Computations:** Both sequentialSSSP and updateSSSP are relatively fast, suggesting that the graph size or update complexity is manageable post-loading.

3. Call Graph Analysis

The call graph provides a hierarchical view of function interactions, showing how time propagates through the call stack.

Key Observations:

- **Main [1] (99.9%, 0.02 self, 59.70 children):**
 - Calls loadGraph, sequentialSSSP, updateSSSP, and others.
 - Most time is in children, particularly loadGraph (35.00 self + 22.35 children).
 - High call count for std::__cxx11::to_string (6,420,370) from output operations.
- **loadGraph [2] (95.9%, 35.00 self, 22.35 children):**
 - Major children:
 - std::insert_unique (18.82 seconds, 10M calls): Edge duplicate detection.
 - std::__introsort_loop (1.50 seconds): Edge sorting.
 - Vector reallocation functions (e.g., 0.81 seconds for Edge insertions).
 - The high child time reflects intensive data structure operations (set insertions, sorting, vector growth).
- **sequentialSSSP [4] (3.9%, 1.07 self, 1.26 children):**
 - Calls frame_dummy (1.26 seconds, 3,757,078 calls), likely a profiling artifact.
 - Minimal other children, indicating Dijkstra's algorithm is self-contained.
- **updateSSSP [12] (0.02%, 0.01 self):**
 - Calls Graph::removeEdge (50 calls) and vector erase operations, but time is negligible.
 - Suggests few edge changes or efficient updates.

4. Performance Bottlenecks and Insights

Primary Bottleneck:

- **Graph Loading (loadGraph):**
 - **Issue:** 58.55% of runtime (35.00 seconds) is spent reading and preprocessing the graph, particularly in std::_Rb_tree::_M_insert_unique (31.48%, 18.82 seconds) for duplicate edge detection.
 - **Cause:** The set<pair<int64_t, int64_t>> used to track unique edges has O(log n) insertion complexity, and 10 million insertions amplify this cost. Additionally, sorting edges (1.50 seconds) and vector reallocations contribute.
 - **Impact:** For large graphs, preprocessing dominates over actual SSSP computations.

Secondary Bottlenecks:

- **Edge Sorting:** std::__introsort_loop (1.50 seconds) sorts edges for CSR format, necessary but costly for large edge sets.
- **Memory Management:** Vector reallocations (e.g., 0.81 seconds for Edge, 0.43 seconds for pair<long, long>) indicate frequent dynamic resizing, which could be optimized.

- **Profiling Overhead:** `frame_dummy` (1.84 seconds, 5.5M calls) suggests `gprof` instrumentation may inflate times, particularly for `loadGraph` and `sequentialSSSP`.

Efficient Components:

- **SSSP Computations:** `sequentialSSSP` (1.79%) and `updateSSSP` (0.02%) are efficient, indicating that Dijkstra's algorithm and dynamic updates scale well for the graph size and update set.
- **Output Operations:** Minimal time in `to_string` and file output, despite high call counts.

6. Conclusion

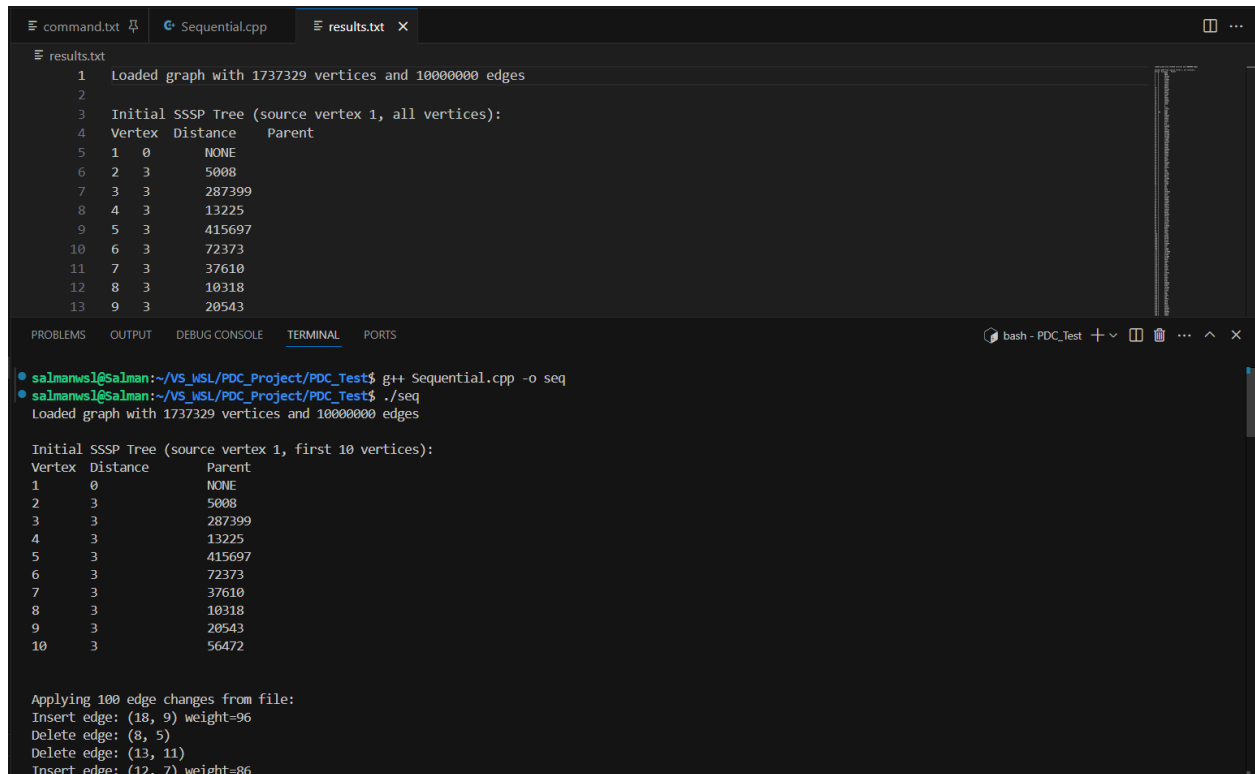
The `Sequential.cpp` program's performance is dominated by graph loading (`loadGraph`, 58.55%), particularly due to duplicate edge detection (`std::_Rb_tree::_M_insert_unique`, 31.48%). The SSSP computations (`sequentialSSSP` and `updateSSSP`) are efficient, contributing only ~2% to runtime. Optimization efforts should focus on replacing the set with a faster data structure (e.g., `unordered_set`), pre-reserving vectors, and exploring parallel or buffered I/O. Additionally, re-profiling with a sampling-based tool could clarify `frame_dummy` overhead. These changes could potentially reduce the total runtime by 20–30 seconds, making the program significantly faster for large graphs.

Key Takeaways:

- **Bottleneck:** Graph preprocessing, especially edge duplicate detection.
- **Efficient Areas:** SSSP algorithms and update handling.

Execution Screenshots

Sequential Implementation



```
command.txt Sequential.cpp results.txt X
results.txt
1 Loaded graph with 1737329 vertices and 10000000 edges
2
3 Initial SSSP Tree (source vertex 1, all vertices):
4 Vertex Distance Parent
5 1 0 NONE
6 2 3 5008
7 3 3 287399
8 4 3 13225
9 5 3 415697
10 6 3 72373
11 7 3 37610
12 8 3 10318
13 9 3 20543

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
bash - PDC_Test + - - - ^ X

salmanwsl@Salman:~/VS_WSL/PDC_Project/PDC_Test$ g++ Sequential.cpp -o seq
salmanwsl@Salman:~/VS_WSL/PDC_Project/PDC_Test$ ./seq
Loaded graph with 1737329 vertices and 10000000 edges

Initial SSSP Tree (source vertex 1, first 10 vertices):
Vertex Distance Parent
1 0 NONE
2 3 5008
3 3 287399
4 3 13225
5 3 415697
6 3 72373
7 3 37610
8 3 10318
9 3 20543
10 3 56472

Applying 100 edge changes from file:
Insert edge: (18, 9) weight=96
Delete edge: (8, 5)
Delete edge: (13, 11)
Insert edge: (12, 7) weight=86
```

Dynamic Algorithm (OpenMP)

```
command.txt  Dynamic_MP.cpp 2  results.txt X
results.txt
1 loaded graph with 1737329 vertices and 10000000 edges
2
3 Initial SSSP Tree (source vertex 1, all vertices):
4 Vertex Distance Parent
5 1 0 NONE
6 2 3 5008
7 3 3 287399
8 4 3 13225
9 5 3 415697
10 6 3 72373
11 7 3 37610
12 8 3 10318
13 9 3 20543

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
bash - PDC_Test + - - - ^ X

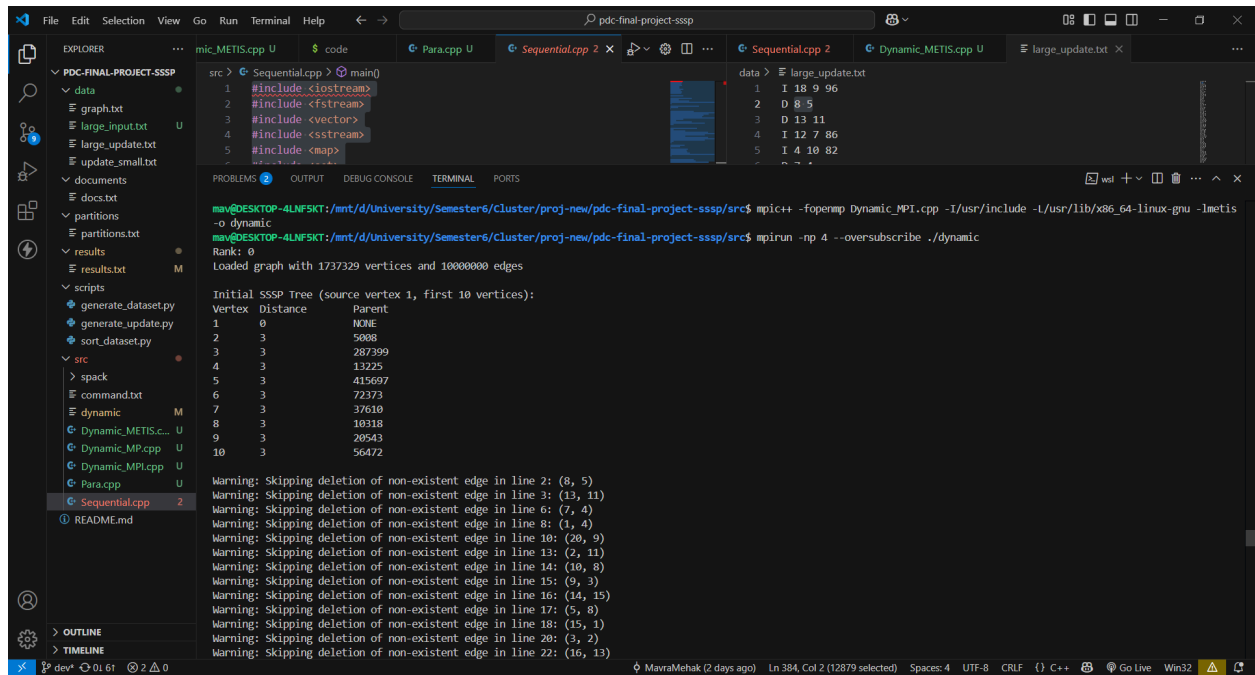
salmanwsl@Salman:~/VS_WSL/PDC_Project/PDC_Test$ g++ -fopenmp Dynamic_MP.cpp -I/usr/include -L/usr/lib/x86_64-linux-gnu -o dynamiccmp
salmanwsl@Salman:~/VS_WSL/PDC_Project/PDC_Test$ ./dynamiccmp

Initial SSSP Tree (source vertex 1, first 10 vertices):
Vertex Distance Parent
1 0 NONE
2 3 5008
3 3 287399
4 3 13225
5 3 415697
6 3 72373
7 3 37610
8 3 10318
9 3 20543
10 3 56472

Warning: Skipping deletion of non-existent edge in line 2: (8, 5)
Warning: Skipping deletion of non-existent edge in line 3: (13, 11)
Warning: Skipping deletion of non-existent edge in line 6: (7, 4)
Warning: Skipping deletion of non-existent edge in line 8: (1, 4)
Warning: Skipping deletion of non-existent edge in line 10: (20, 9)
Warning: Skipping deletion of non-existent edge in line 13: (2, 11)
Warning: Skipping deletion of non-existent edge in line 14: (10, 8)

Ln 1, Col 1 Spaces: 4 UTF-8 LF Plain Text
```

Dynamic Algorithm (MPI and OpenMP)



```
src > Sequential.cpp > main()
1 #include <iostream>
2 #include <fstream>
3 #include <vector>
4 #include <sstream>
5 #include <map>

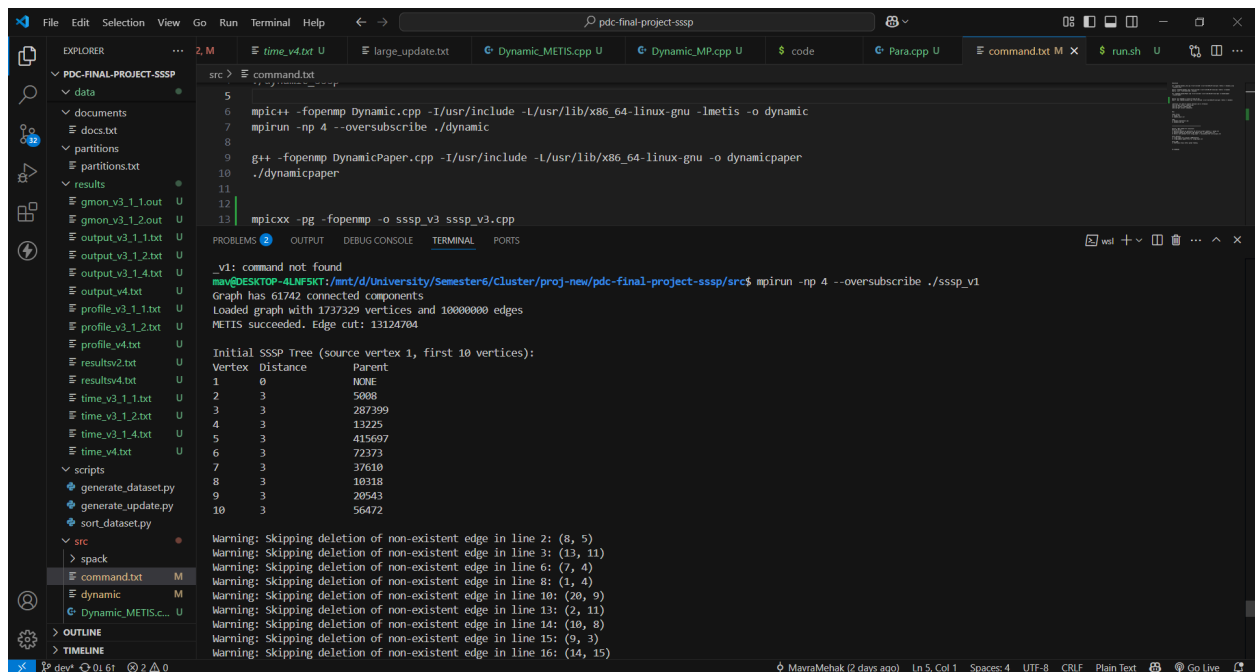
...

mav@DESKTOP-4LNF5KT:/mnt/d/University/Semesters6/Cluster/proj-new/pdc-final-project-sssp/src$ mpic++ -fopenmp Dynamic_MPI.cpp -I/usr/include -I/usr/lib/x86_64-linux-gnu -lmetis -o dynamic
mav@DESKTOP-4LNF5KT:/mnt/d/University/Semesters6/Cluster/proj-new/pdc-final-project-sssp/src$ mpirun -np 4 --oversubscribe ./dynamic
Rank: 0
Loaded graph with 1737329 vertices and 10000000 edges

Initial SSSP Tree (source vertex 1, first 10 vertices):
Vertex Distance Parent
1 0 NONE
2 3 5008
3 3 287399
4 3 13225
5 3 415697
6 3 72373
7 3 37610
8 3 10318
9 3 20543
10 3 56472

Warning: Skipping deletion of non-existent edge in line 2: (8, 5)
Warning: Skipping deletion of non-existent edge in line 3: (13, 11)
Warning: Skipping deletion of non-existent edge in line 6: (7, 4)
Warning: Skipping deletion of non-existent edge in line 8: (1, 4)
Warning: Skipping deletion of non-existent edge in line 10: (20, 9)
Warning: Skipping deletion of non-existent edge in line 13: (2, 11)
Warning: Skipping deletion of non-existent edge in line 14: (10, 8)
Warning: Skipping deletion of non-existent edge in line 15: (9, 3)
Warning: Skipping deletion of non-existent edge in line 16: (14, 15)
Warning: Skipping deletion of non-existent edge in line 17: (5, 8)
Warning: Skipping deletion of non-existent edge in line 18: (15, 1)
Warning: Skipping deletion of non-existent edge in line 20: (3, 2)
Warning: Skipping deletion of non-existent edge in line 22: (16, 13)
```

Dynamic Algorithm (METIS)



```
src > command.txt
5
6 mpic++ -fopenmp Dynamic.cpp -I/usr/include -I/usr/lib/x86_64-linux-gnu -lmetis -o dynamic
7 mpirun -np 4 --oversubscribe ./dynamic
8
9 g++ -fopenmp DynamicPaper.cpp -I/usr/include -I/usr/lib/x86_64-linux-gnu -o dynamicpaper
10 ./dynamicpaper
11
12
13 mpicxx -pg -fopenmp -o sssp_v3 sssp_v3.cpp

...

mav@DESKTOP-4LNF5KT:/mnt/d/University/Semesters6/Cluster/proj-new/pdc-final-project-sssp/src$ mpirun -np 4 --oversubscribe ./sssp_v1
Graph has 61742 connected components
Loaded graph with 1737329 vertices and 10000000 edges
METIS succeeded. Edge cut: 13124704

Initial SSSP Tree (source vertex 1, first 10 vertices):
Vertex Distance Parent
1 0 NONE
2 3 5008
3 3 287399
4 3 13225
5 3 415697
6 3 72373
7 3 37610
8 3 10318
9 3 20543
10 3 56472

Warning: Skipping deletion of non-existent edge in line 2: (8, 5)
Warning: Skipping deletion of non-existent edge in line 3: (13, 11)
Warning: Skipping deletion of non-existent edge in line 6: (7, 4)
Warning: Skipping deletion of non-existent edge in line 8: (1, 4)
Warning: Skipping deletion of non-existent edge in line 10: (20, 9)
Warning: Skipping deletion of non-existent edge in line 13: (2, 11)
Warning: Skipping deletion of non-existent edge in line 14: (10, 8)
Warning: Skipping deletion of non-existent edge in line 15: (9, 3)
Warning: Skipping deletion of non-existent edge in line 16: (14, 15)
```